

Esercizio 3

Costruire un **albero binario di ricerca** inserendo *successivamente* (uno dopo l'altro) i seguenti valori:

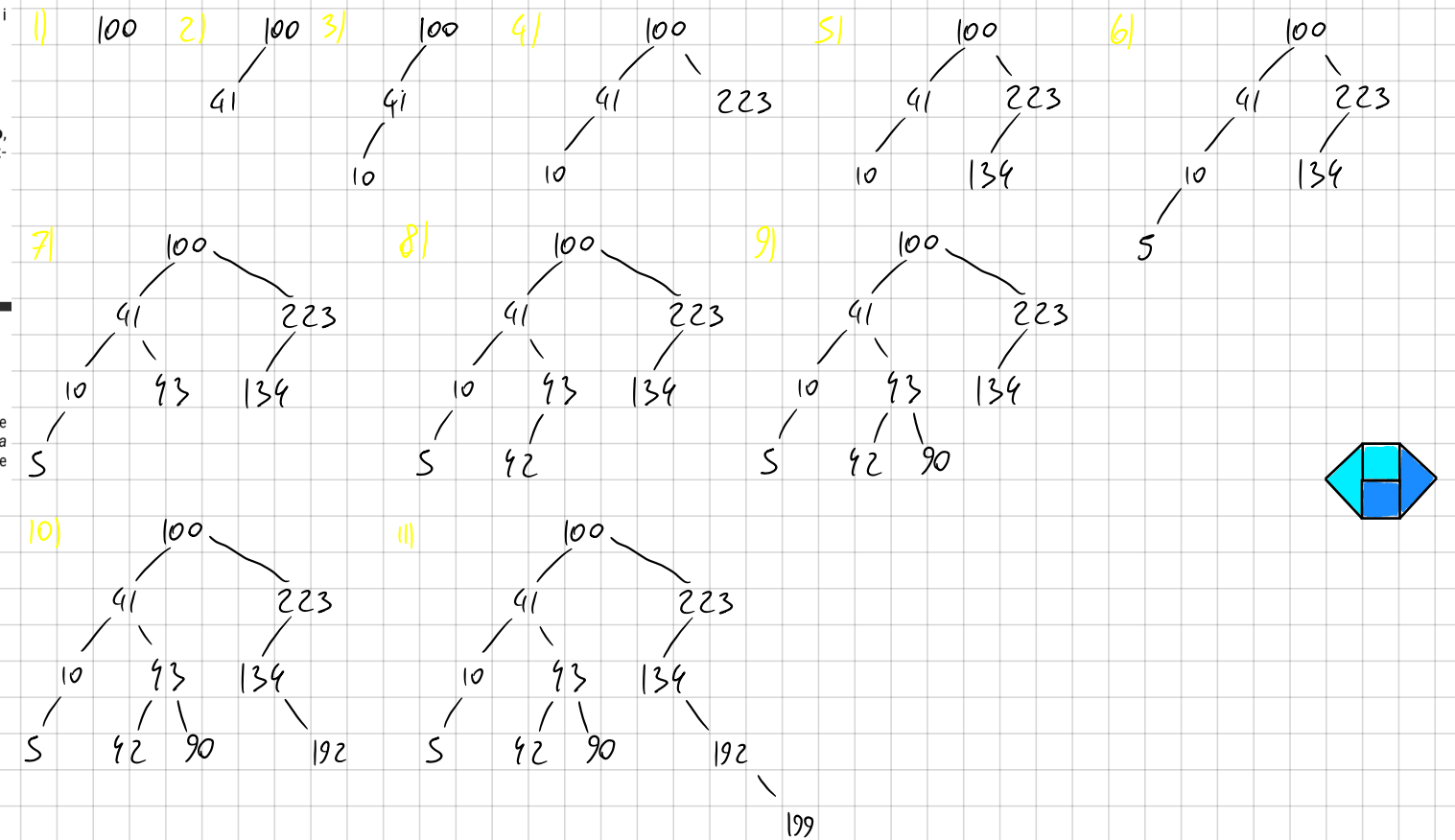
100, 41, 10, 223, 134, 5, 43, 42, 90, 192, 199

Disegnare gli alberi ottenuti **dopo ogni inserimento**.

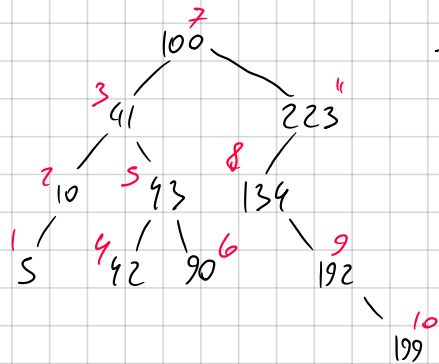
Si mostrino poi:

- le sequenze di interi ottenute visitando l'albero finale negli ordini **anticipato**, **simmetrico** e **posticipato** (ovvero, rispettivamente: pre-ordine, in-ordine e post-ordine);

- gli alberi ottenuti cancellando prima la chiave 90, poi la chiave 134 e infine la chiave 41 (mostrare la struttura dell'albero ottenuta dopo *ogni singola cancellazione*, a partire dall'albero precedente). Spiegare brevemente come vengono fatte le operazioni di cancellazione.

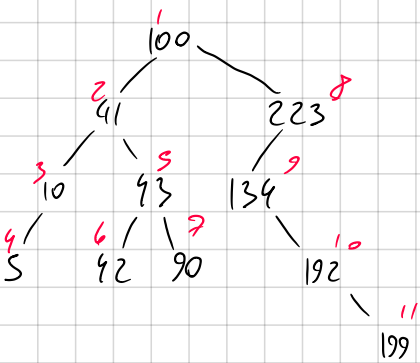


Inorder visit (x - centro - dx)



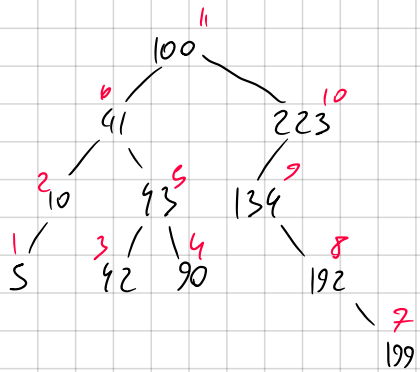
5 10 41 42 43 90 100 134 192 199 223

Preorder visit (Centro - dx - dx)



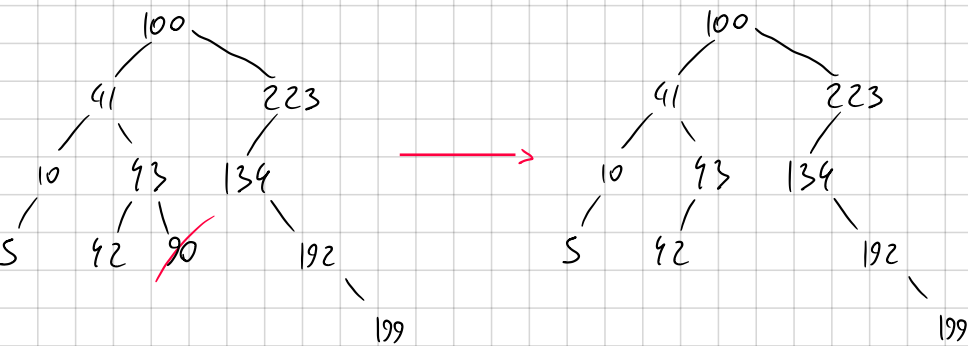
100 41 10 5 43 42 90 223 134 192 199

Postorder visit (dx - dx - Centro)



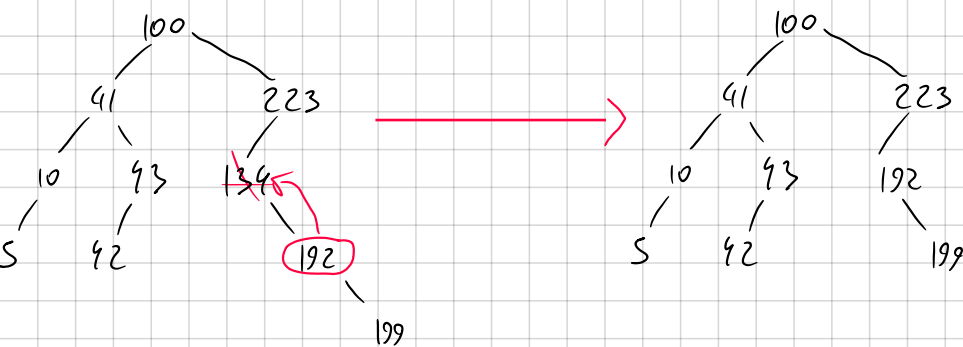
5 10 42 90 43 41 199 192 134 223 100

1) Cancello il 90



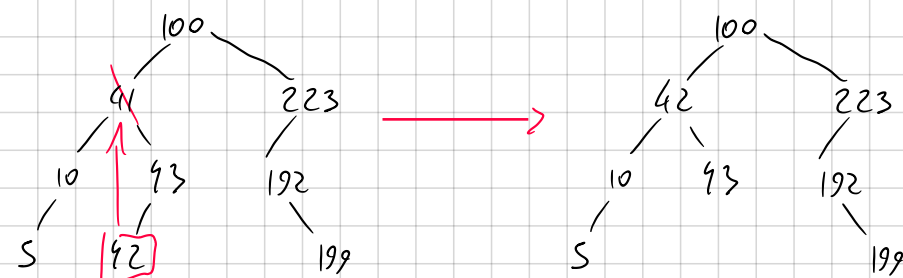
È una semplice foglia quindi lo posso cancellare senza modificare ulteriormente la struttura dell'albero

2) Cancello il 134



Essendo che il nonno ha un solo figlio, dopo la cancellazione lo sostituisco col figlio

3) Cancello il 41



Essendo che il nodo 41 ha 2 figli lo sostituisco col suo successore ossia il 42 dopo averlo sostituito posto il 43 come figlio destro del 42

Esercizio 1

Scrivere un algoritmo **iterativo** che, dati in ingresso:

- una **lista concatenata doppia** L , i cui nodi contengono un campo *key* (contenente un carattere dell'alfabeto italiano), un campo *next* (puntatore al prossimo elemento della lista) e un campo *prev* (puntatore all'elemento precedente nella lista);
- un carattere CAR ,

Che scrittura
da sinistra

elimini dalla lista (senza distruggerla) tutti i nodi il cui campo *key* è uguale a CAR .

Valutare poi i **tempi di calcolo** nei casi *migliore* e *peggiore*, indicando quante volte viene eseguita ogni operazione in pseudocodice. Si assuma che ogni singola operazione elementare impieghi lo stesso tempo (costante).

procedura 1

$current = L.head$

while $current \neq null$ // Itera attraverso la lista finché

if $current.key == CAR$

if $current.next \neq null$ // Se current non è l'ultimo nodo, il suo next deve puntare al prev del nodo precedente

$current.next.prev = current.prev$

else // Se current è l'ultimo nodo (code), aggiorna il puntatore tail della lista

$L.tail = current.prev$

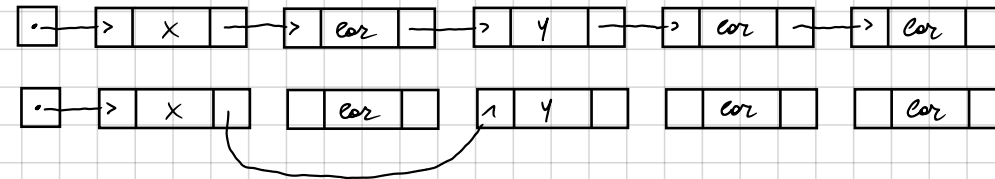
if $current.prev \neq null$ // Se current non è il primo nodo aggiorna il puntatore head della lista

$current.prev.next = current.next$

else

$L.head = current.next$

$current = current.next$



Esercizio 2

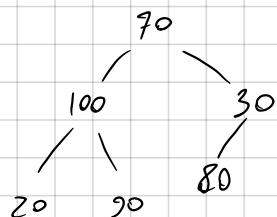
Simulare l'esecuzione dell'algoritmo HEAPSORT sul seguente array:

$A = [70, 100, 30, 20, 90, 80]$

illustrando i vari passaggi, e mostrando lo stato dell'array **dopo ogni operazione di scambio** fra i suoi elementi. In particolare, si mostrino anzitutto gli scambi effettuati dalla funzione BUILDHEAP per risistemare gli elementi del vettore in modo da formare uno heap, e successivamente si mostrino gli scambi effettuati dall'algoritmo di ordinamento vero e proprio per ordinare gli elementi in ordine crescente. Se lo si ritiene opportuno, l'array può essere rappresentato in forma di albero; in tal caso, sul primo albero si indichino in maniera chiara gli indici degli elementi.

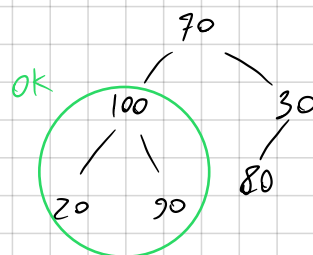
1)

$A = [70, 100, 30, 20, 90, 80]$



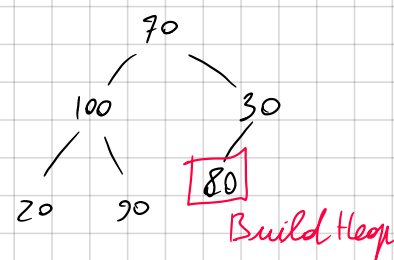
2)

$A = [70, 100, 30, 20, 90, 80]$



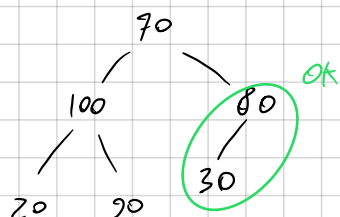
3)

$A = [70, 100, 30, 20, 90, 80]$



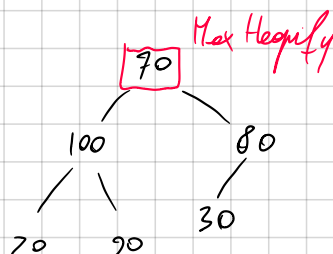
4)

$A = [70, 100, 80, 20, 90, 30]$



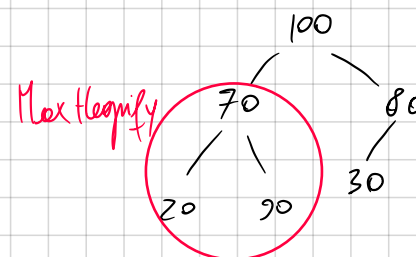
5)

$A = [70, 100, 80, 20, 90, 30]$



6)

$A = [100, 70, 80, 20, 90, 30]$



7)

$A = [100, 90, 80, 20, 70, 30]$

